

《AI Agent数据治理实战手册》

当下企业数字化转型中，数据治理是核心刚需，而传统人工治理模式耗时耗力、误差率高，难以适配PB级多模态数据的处理需求。AI Agent的出现，彻底打破这一困境，实现数据治理全流程自动化、标准化。

本文为纯实战手册，摒弃冗余理论，聚焦拿来就用，整合AI Agent数据治理全流程实操指南、可直接复制的Prompt库、可运行的轻量化代码库，覆盖从环境搭建到场景落地的每一个关键环节，无论是技术从业者、数据分析师，还是中小企业管理者，收藏本文即可快速落地，无需再翻阅其他资料。

第一章：手册核心定位与适用人群

1.1 核心定位

本手册以实战落地为核心，无需自研框架、无需复杂配置，依托开源工具，搭配现成Prompt和代码，帮助用户快速搭建AI Agent数据治理体系，解决数据清洗慢、标注成本高、合规难管控三大核心痛点，实现效率翻倍。

1.2 适用人群

- 技术从业者（开发、测试、数据工程师）：快速复用代码、Prompt，落地AI Agent数据治理场景；
- 数据相关岗位（数据分析师、数据运营）：借助AI Agent简化数据处理流程，减少人工干预；
- 中小企业管理者：无需投入高额研发成本，轻量化落地数据治理，提升企业数据价值；
- AI新手：快速入门AI Agent在数据治理中的应用，掌握实操技巧。

第二章：前置环境搭建（10分钟完成，零门槛）

无需专业运维能力，按照以下步骤搭建基础环境，所有工具均为开源免费，适配Windows、Linux、Mac多系统，新手可直接照搬操作。

2.1 必备工具与安装指令

核心依赖：Python环境、Agent框架、数据处理工具、开源大模型，安装指令可直接复制运行，无需修改。

- 1 # 1. 安装Python (3.8-3.10版本, 兼容性最佳)
- 2 # Windows/Mac: 直接从官网下载安装 <https://www.python.org/downloads/>

```

3  # Linux:
4  sudo apt update
5  sudo apt install python3.9 python3-pip
6
7  # 2. 安装核心依赖包 (Agent框架+数据处理+大模型调用)
8  pip install langchain pandas pymysql openai transformers accelerate
   sentencepiece
9
10 # 3. 下载开源大模型 (本地运行, 无需付费, 适配中小场景)
11 # 推荐模型: Qwen-7B-Chat (轻量高效)、Llama 3-8B-Instruct
12 # 下载地址: https://modelscope.cn/models (搜索对应模型, 按指引下载解压, 记录模型路
   径)

```

2.2 基础配置 (复制即用, 仅修改2处)

配置文件用于统一管理模型路径、数据存储路径等核心参数, 修改后可全局复用, 无需重复配置。

```

1  # AI Agent数据治理基础配置文件 (config.py)
2  # 直接复制保存为config.py, 仅修改以下2处核心参数
3  class AIAgentDataGovernConfig:
4      # 1. 修改为你下载的开源大模型路径 (解压后的文件夹路径)
5      LLM_MODEL_PATH = "D:/models/qwen-7b-chat" # Windows示例
6      # LLM_MODEL_PATH = "/home/user/models/llama-3-8b-instruct" # Linux示例
7
8      # 2. 修改为数据存储路径 (用于存放原始数据、处理后数据)
9      DATA_STORAGE_PATH = "D:/ai_agent_data" # Windows示例
10     # DATA_STORAGE_PATH = "/home/user/ai_agent_data" # Linux示例
11
12     # 以下参数无需修改, 直接套用
13     BATCH_PROCESS_SIZE = 5000 # 批量处理数据量 (根据电脑配置调整)
14     CONTEXT_LENGTH = 2048 # 大模型上下文长度
15     DATA_QUALITY_TARGET = 0.99 # 数据治理目标准确率
16     MANUAL_INTERVENTION_RATE = 0.01 # 人工干预率目标 (≤1%)
17
18     # 初始化配置, 后续代码直接调用
19     config = AIAgentDataGovernConfig()

```

2.3 环境校验 (确保可正常运行)

运行以下代码, 校验环境是否搭建成功, 若输出环境校验通过, 则可进入后续实战环节。

```

1  # 环境校验代码 (直接复制运行)
2  from config import config
3  from langchain.llms import LlamaCpp

```

```

4
5  try:
6      # 初始化大模型
7      llm = LlamaCpp(
8          model_path=config.LLM_MODEL_PATH,
9          n_ctx=config.CONTEXT_LENGTH,
10         temperature=0.1
11     )
12     # 测试大模型响应
13     test_response = llm("请简单介绍AI Agent在数据治理中的作用")
14     print("环境校验通过! ")
15     print("大模型测试响应: ", test_response[:50]) # 输出前50字符, 验证可用性
16 except Exception as e:
17     print(f"环境校验失败, 错误原因: {str(e)}")
18     print("请检查模型路径是否正确、依赖包是否安装完整")

```

第三章：核心实战场景（含Prompt+代码，直接复用）

聚焦企业数据治理最高频的3个场景，每个场景均包含场景说明+Prompt库（可直接复制）+完整代码（可运行）+注意事项，无需额外调试，修改路径即可落地。

场景1：结构化数据自动化清洗（高频刚需）

场景说明

适用：CSV、Excel、数据库（MySQL、Oracle）等结构化数据，解决重复值、缺失值、格式错误、异常值（如超出合理范围的数值）等问题，批量处理效率较人工提升10倍以上。

可直接复制Prompt库

3个Prompt覆盖问题识别→清洗执行→效果校验全流程，替换{data_sample}、{data_issues}等占位符即可使用。

```

1  # Prompt1: 数据问题识别（自动排查所有清洗痛点）
2  角色：专业AI数据清洗Agent，严谨高效，仅输出结构化结果，不添加多余解释。
3  任务：识别以下数据样本中的所有问题，包括但不限于重复值、缺失值、格式错误（日期、数值、文本）、异常值，按问题类型|涉及列名|具体描述的格式输出，确保无遗漏。
4  数据样本：{data_sample}
5
6  # Prompt2: 清洗指令生成（自动生成可执行代码）
7  角色：AI数据清洗Agent，精通pandas数据处理，生成的代码可直接运行，无需修改。
8  任务：根据以下识别出的数据问题，生成对应的pandas清洗代码，要求清洗后数据格式统一、无异常，代码末尾需返回清洗后的数据。
9  数据问题：{data_issues}
10

```

```
11 # Prompt3: 清洗效果校验 (自动验证结果)
12 角色: AI数据质量校验Agent, 严格核对清洗效果, 仅输出校验报告。
13 任务: 对比原始数据问题和清洗后的数据, 输出校验报告, 包含清洗完成率、数据准确率、未解决问题 (若有), 若有未解决问题, 补充优化代码。
14 原始数据问题: {data_issues}
15 清洗后数据样本: {cleaned_data_sample}
```

可直接运行代码库

完整代码包含数据加载、问题识别、自动清洗、效果校验、结果保存, 仅修改main函数中的数据路径即可运行。

```
1 # 结构化数据自动化清洗完整代码 (clean_data_agent.py)
2 import pandas as pd
3 from langchain.llms import LlamaCpp
4 from langchain.prompts import PromptTemplate
5 from config import config
6
7 # 1. 初始化大模型 (无需修改)
8 llm = LlamaCpp(
9     model_path=config.LLM_MODEL_PATH,
10     n_ctx=config.CONTEXT_LENGTH,
11     temperature=0.1 # 降低随机性, 确保结果精准
12 )
13
14 # 2. 加载Prompt模板 (直接复用, 无需修改)
15 # 数据问题识别Prompt
16 issue_detect_prompt = PromptTemplate(
17     input_variables=["data_sample"],
18     template="角色: 专业AI数据清洗Agent, 严谨高效, 仅输出结构化结果, 不添加多余解释。
19     任务: 识别以下数据样本中的所有问题, 包括但不限于重复值、缺失值、格式错误 (日期、数值、文本)、异常值, 按问题类型|涉及列名|具体描述的格式输出, 确保无遗漏。
20     数据样本: {data_sample}"
21 )
22
23 # 清洗指令生成Prompt
24 clean_code_prompt = PromptTemplate(
25     input_variables=["data_issues"],
26     template="角色: AI数据清洗Agent, 精通pandas数据处理, 生成的代码可直接运行, 无需修改。
27     任务: 根据以下识别出的数据问题, 生成对应的pandas清洗代码, 要求清洗后数据格式统一、无异常, 代码末尾需返回清洗后的数据。
28     数据问题: {data_issues}"
29 )
30
```

```

31 # 清洗效果校验Prompt
32 check_effect_prompt = PromptTemplate(
33     input_variables=["data_issues", "cleaned_data_sample"],
34     template="角色：AI数据质量校验Agent，严格核对清洗效果，仅输出校验报告。
35     任务：对比原始数据问题和清洗后的数据，输出校验报告，包含清洗完成率、数据准确率、未解决问题（若有），若有未解决问题，补充优化代码。
36     原始数据问题：{data_issues}
37     清洗后数据样本：{cleaned_data_sample}"
38 )
39
40 # 3. 核心功能函数（无需修改）
41 def load_data(data_path):
42     """加载原始数据，支持CSV、Excel、MySQL数据库"""
43     if data_path.endswith(".csv"):
44         return pd.read_csv(data_path, encoding="utf-8")
45     elif data_path.endswith(".xlsx"):
46         return pd.read_excel(data_path)
47     elif "mysql://" in data_path:
48         # 适配MySQL数据库（示例：mysql://user:password@host:port/dbname?
49         # table=table_name）
50         import pymysql
51         from sqlalchemy import create_engine
52         engine = create_engine(data_path)
53         return pd.read_sql_table(table_name=data_path.split("table=")[1],
54                                 con=engine)
55     else:
56         raise ValueError("仅支持CSV、Excel、MySQL格式数据")
57
58 def detect_data_issues(data):
59     """自动识别数据问题"""
60     # 取前100条样本识别问题（兼顾效率与准确性）
61     data_sample = data.head(100).to_string()
62     issues = llm(issue_detect_prompt.format(data_sample=data_sample))
63     return issues
64
65 def auto_clean_data(data, issues):
66     """自动执行数据清洗"""
67     # 生成清洗代码并执行
68     clean_code = llm(clean_code_prompt.format(data_issues=issues))
69     exec(clean_code, globals(), locals())
70     return locals()["cleaned_data"] # 返回清洗后的数据
71
72 def check_clean_effect(issues, cleaned_data):
73     """校验清洗效果，生成校验报告"""
74     cleaned_sample = cleaned_data.head(100).to_string()
75     check_report = llm(check_effect_prompt.format(
76         data_issues=issues,

```

```

75         cleaned_data_sample=cleaned_sample
76     ))
77     return check_report
78
79 # 4. 主函数 (仅修改2处数据路径)
80 def main():
81     # 需修改1: 原始数据路径 (CSV/Excel/MySQL)
82     raw_data_path = f"{config.DATA_STORAGE_PATH}/raw_data.csv"
83     # 需修改2: 清洗后数据输出路径
84     output_data_path = f"{config.DATA_STORAGE_PATH}/cleaned_data.csv"
85
86     # 执行清洗流程 (无需修改)
87     try:
88         data = load_data(raw_data_path)
89         print("数据加载成功, 开始识别数据问题...")
90         issues = detect_data_issues(data)
91         print("数据问题识别完成, 开始自动清洗...")
92         cleaned_data = auto_clean_data(data, issues)
93         print("数据清洗完成, 开始校验效果...")
94         check_report = check_clean_effect(issues, cleaned_data)
95
96         # 保存清洗后的数据
97         cleaned_data.to_csv(output_data_path, index=False, encoding="utf-8")
98         print("\n=== 清洗完成 ===")
99         print(f"清洗后数据已保存至: {output_data_path}")
100        print("\n校验报告: ")
101        print(check_report)
102    except Exception as e:
103        print(f"执行失败, 错误原因: {str(e)}")
104
105 if __name__ == "__main__":
106     main()

```

注意事项

- 若数据量较大（超过10万条），可调整config.py中的BATCH_PROCESS_SIZE参数，减小批量处理量，避免内存不足；
- 数据库数据加载时，需确保MySQL账号密码正确，且具备对应数据表的读取权限；
- 清洗后的文件默认保存为CSV格式，如需Excel格式，将to_csv改为to_excel即可。

场景2：多模态数据自动化标注（适配大模型训练）

场景说明

适用：文本（用户反馈、知识库文档、评论）、音频（客服对话、语音指令）、图像（产品图片、场景图片）标注，解决人工标注成本高、精度低、效率慢的痛点，标注结果可直接用于大模型训练/微调。

可直接复制Prompt库

适配3种模态数据，每个Prompt均明确标注规则，替换占位符即可直接使用，无需额外调整。

```
1  # Prompt1: 文本数据标注（情感分类+关键词提取，适配用户反馈、评论）
2  角色：AI文本标注Agent，精准高效，严格按照标注规则执行，仅输出结构化标注结果。
3  标注规则：1. 情感分类：分为正面、负面、中性三类；2. 关键词提取：提取与核心问题、需求相关的3-5个关键词；3. 输出格式：文本内容|情感标签|关键词（用逗号分隔）。
4  任务：按照标注规则，对以下文本数据进行标注，无需多余解释，直接输出结果。
5  文本数据：{text_data}
6
7  # Prompt2: 音频数据标注（关键信息提取，适配客服对话）
8  角色：AI音频标注Agent，先将音频转文字，再进行关键信息提取，仅输出结构化结果。
9  标注规则：1. 先将音频转写为文本；2. 提取关键信息：用户问题、需求、解决方案、客服回复重点；3. 输出格式：音频转写文本|用户问题|用户需求|解决方案。
10 任务：对以下音频数据进行转写和标注，无需多余解释，直接输出结果。
11 音频数据（base64格式或音频路径）：{audio_data}
12
13 # Prompt3: 图像数据标注（目标检测+分类，适配产品图片）
14 角色：AI图像标注Agent，识别图像中的目标，进行分类和边界框标注，仅输出结构化结果。
15 标注规则：1. 目标分类：按照{image_category}进行分类；2. 边界框标注：输出目标的x1,y1,x2,y2坐标（左上角和右下角）；3. 输出格式：图像路径|目标分类|边界框坐标|置信度。
16 任务：对以下图像数据进行标注，无需多余解释，直接输出结果。
17 图像数据（路径或base64格式）：{image_data}
18 图像分类规则：{image_category}
```

可直接运行代码库

支持多模态数据标注，整合文本、音频、图像处理逻辑，修改数据路径和标注规则即可运行。

```
1  # 多模态数据自动化标注完整代码 (annotation_agent.py)
2  import os
3  import base64
4  from langchain.llms import LlamaCpp
5  from langchain.prompts import PromptTemplate
6  from config import config
7  from PIL import Image
8  import speech_recognition as sr
9
10 # 1. 初始化大模型（无需修改）
11 llm = LlamaCpp(
```

```
12     model_path=config.LLM_MODEL_PATH,
13     n_ctx=config.CONTEXT_LENGTH,
14     temperature=0.1
15 )
16
17 # 2. 加载Prompt模板 (直接复用, 无需修改)
18 # 文本标注Prompt
19 text_annotation_prompt = PromptTemplate(
20     input_variables=["text_data"],
21     template="角色: AI文本标注Agent, 精准高效, 严格按照标注规则执行, 仅输出结构化标注
22     结果。
23     标注规则: 1. 情感分类: 分为正面、负面、中性三类; 2. 关键词提取: 提取与核心问题、需求相关的
24     3-5个关键词; 3. 输出格式: 文本内容|情感标签|关键词 (用逗号分隔)。
25     任务: 按照标注规则, 对以下文本数据进行标注, 无需多余解释, 直接输出结果。
26     文本数据: {text_data}"
27 )
28
29 # 音频标注Prompt
30 audio_annotation_prompt = PromptTemplate(
31     input_variables=["audio_text"],
32     template="角色: AI音频标注Agent, 已完成音频转写, 仅进行关键信息提取和结构化输出。
33     标注规则: 1. 提取关键信息: 用户问题、需求、解决方案、客服回复重点; 2. 输出格式: 音频转写
34     文本|用户问题|用户需求|解决方案。
35     任务: 对以下音频转写文本进行标注, 无需多余解释, 直接输出结果。
36     音频转写文本: {audio_text}"
37 )
38
39 # 图像标注Prompt
40 image_annotation_prompt = PromptTemplate(
41     input_variables=["image_info", "image_category"],
42     template="角色: AI图像标注Agent, 识别图像中的目标, 进行分类和边界框标注, 仅输出结
43     构化结果。
44     标注规则: 1. 目标分类: 按照{image_category}进行分类; 2. 边界框标注: 输出目标的
45     x1,y1,x2,y2坐标 (左上角和右下角); 3. 输出格式: 图像路径|目标分类|边界框坐标|置信度。
46     任务: 对以下图像数据进行标注, 无需多余解释, 直接输出结果。
47     图像信息: {image_info}
48     图像分类规则: {image_category}"
49 )
50
51 # 3. 多模态数据处理辅助函数 (无需修改)
52 def process_text_data(text_path):
53     """加载文本数据, 返回文本列表"""
54     with open(text_path, "r", encoding="utf-8") as f:
55         return [line.strip() for line in f if line.strip()]
56
57 def process_audio_data(audio_path):
58     """将音频转写为文本"""
```

```
54     r = sr.Recognizer()
55     with sr.AudioFile(audio_path) as source:
56         audio = r.record(source)
57     try:
58         return r.recognize_google(audio, language="zh-CN")
59     except:
60         return "音频转写失败, 请检查音频文件"
61
62 def process_image_data(image_path):
63     """获取图像基础信息, 用于标注"""
64     img = Image.open(image_path)
65     width, height = img.size
66     return f"图像路径: {image_path}, 尺寸: {width}x{height}, 图像格式: {img.format}"
67
68 # 4. 核心标注函数 (无需修改)
69 def text_annotation(text_data):
70     """文本数据标注"""
71     annotations = []
72     for text in text_data:
73         if not text:
74             continue
75         annotation = llm(text_annotation_prompt.format(text_data=text))
76         annotations.append(annotation)
77     return annotations
78
79 def audio_annotation(audio_path):
80     """音频数据标注"""
81     audio_text = process_audio_data(audio_path)
82     if "转写失败" in audio_text:
83         return [f"{audio_path}|转写失败|无|无|无"]
84     annotation = llm(audio_annotation_prompt.format(audio_text=audio_text))
85     return [f"{audio_path}|{annotation}"]
86
87 def image_annotation(image_path, image_category):
88     """图像数据标注"""
89     image_info = process_image_data(image_path)
90     annotation = llm(image_annotation_prompt.format(
91         image_info=image_info,
92         image_category=image_category
93     ))
94     return [annotation]
95
96 # 5. 主函数 (修改数据路径和标注规则即可)
97 def main():
98     # 需修改1: 标注类型 (text/audio/image)
99     annotation_type = "text"
```

```
100     # 需修改2: 数据路径 (文本/音频/图像文件夹或单个文件)
101     data_path = f"{config.DATA_STORAGE_PATH}/text_data.txt"
102     # 需修改3: 图像标注专用 (若为image类型, 填写分类规则; 其他类型无需修改)
103     image_category = "产品、背景、文字、logo"
104
105     # 执行标注流程 (无需修改)
106     try:
107         annotations = []
108         if annotation_type == "text":
109             text_data = process_text_data(data_path)
110             annotations = text_annotation(text_data)
111         elif annotation_type == "audio":
112             # 若为文件夹, 遍历所有音频文件
113             if os.path.isdir(data_path):
114                 for file in os.listdir(data_path):
115                     if file.endswith((".wav", ".mp3")):
116                         audio_file = os.path.join(data_path, file)
117                         annotations.extend(audio_annotation(audio_file))
118             else:
119                 annotations = audio_annotation(data_path)
120         elif annotation_type == "image":
121             # 若为文件夹, 遍历所有图像文件
122             if os.path.isdir(data_path):
123                 for file in os.listdir(data_path):
124                     if file.endswith((".jpg", ".png", ".jpeg")):
125                         image_file = os.path.join(data_path, file)
126                         annotations.extend(image_annotation(image_file,
127 image_category))
128             else:
129                 annotations = image_annotation(data_path, image_category)
130         else:
131             raise ValueError("标注类型仅支持text、audio、image")
132
133         # 保存标注结果
134         output_path = f"
{config.DATA_STORAGE_PATH}/{annotation_type}_annotations.txt"
135         with open(output_path, "w", encoding="utf-8") as f:
136             f.write("\n".join(annotations))
137
138         print(f"标注完成! 共标注{len(annotations)}条数据, 结果已保存至:
{output_path}")
139     except Exception as e:
140         print(f"标注失败, 错误原因: {str(e)}")
141
142 if __name__ == "__main__":
143     main()
```

注意事项

- 音频标注需确保音频文件为wav、mp3格式，且声音清晰，避免背景噪音过大导致转写失败；
- 图像标注的分类规则可根据自身业务调整，例如电商场景可改为商品、包装、模特、背景；
- 标注结果默认保存为txt格式，如需Excel格式，可修改保存逻辑，添加pandas相关代码。

场景3：数据合规审计+血缘追踪（企业必备）

场景说明

适用：企业跨部门数据流转、敏感数据管控、合规审计场景，实现敏感数据自动识别、数据血缘全链路追踪、合规报告自动生成，满足行业监管要求，降低合规风险。

可直接复制Prompt库

覆盖敏感数据识别、血缘追踪、合规报告生成，适配多数行业合规要求，可直接复用。

```
1  # Prompt1: 敏感数据识别（自动识别合规风险）
2  角色：AI合规审计Agent，熟悉行业合规规则，精准识别敏感数据，仅输出结构化结果。
3  合规规则：敏感数据包括身份证号、手机号、银行卡号、邮箱、住址、企业机密信息等。
4  任务：识别以下数据中的敏感数据，按敏感数据类型|所在列|具体内容（脱敏处理）的格式输出，无需多余解释。
5  数据样本：{data_sample}
6
7  # Prompt2: 数据血缘追踪（全链路自动梳理）
8  角色：AI数据血缘追踪Agent，自动梳理数据流转链路，仅输出结构化血缘图谱。
9  任务：根据以下数据的来源、处理记录，梳理数据全链路流转路径，输出数据来源→处理节点→处理方式→数据去向的结构化结果，无需多余解释。
10 数据信息：{data_info}
11 处理记录：{process_records}
12
13 # Prompt3: 合规审计报告生成（自动生成标准化报告）
14 角色：AI合规审计Agent，根据敏感数据识别结果和血缘追踪结果，生成标准化合规审计报告。
15 任务：生成合规审计报告，包含报告标题、审计范围、敏感数据识别结果、血缘链路完整性、合规结论、整改建议（若有），格式规范，可直接用于监管提交。
16 敏感数据识别结果：{sensitive_data_result}
17 数据血缘追踪结果：{lineage_result}
```

可直接运行代码库

完整代码实现敏感数据识别、血缘追踪、合规报告自动生成，适配企业跨部门数据治理场景。

```
1  # 数据合规审计+血缘追踪完整代码 (compliance_agent.py)
2  import pandas as pd
3  from langchain.llms import LlamaCpp
```

```

4 from langchain.prompts import PromptTemplate
5 from config import config
6
7 # 1. 初始化大模型 (无需修改)
8 llm = LlamaCpp(
9     model_path=config.LLM_MODEL_PATH,
10     n_ctx=config.CONTEXT_LENGTH,
11     temperature=0.1
12 )
13
14 # 2. 加载Prompt模板 (直接复用, 无需修改)
15 # 敏感数据识别Prompt
16 sensitive_data_prompt = PromptTemplate(
17     input_variables=["data_sample"],
18     template="角色: AI合规审计Agent, 熟悉行业合规规则, 精准识别敏感数据, 仅输出结构化结果。
19 合规规则: 敏感数据包括身份证号、手机号、银行卡号、邮箱、住址、企业机密信息等。
20 任务: 识别以下数据中的敏感数据, 按敏感数据类型|所在列|具体内容(脱敏处理)的格式输出, 无需多余解释。
21 数据样本: {data_sample}"
22 )
23
24 # 数据血缘追踪Prompt
25 data_lineage_prompt = PromptTemplate(
26     input_variables=["data_info", "process_records"],
27     template="角色: AI数据血缘追踪Agent, 自动梳理数据流转链路, 仅输出结构化血缘图谱。
28 任务: 根据以下数据的来源、处理记录, 梳理数据全链路流转路径, 输出数据来源→处理节点→处理方式→数据去向的结构化结果, 无需多余解释。
29 数据信息: {data_info}
30 处理记录: {process_records}"
31 )
32
33 # 合规审计报告生成Prompt
34 compliance_report_prompt = PromptTemplate(
35     input_variables=["sensitive_data_result", "lineage_result"],
36     template="角色: AI合规审计Agent, 根据敏感数据识别结果和血缘追踪结果, 生成标准化合规审计报告。
37 任务: 生成合规审计报告, 包含报告标题、审计范围、敏感数据识别结果、血缘链路完整性、合规结论、整改建议(若有), 格式规范, 可直接用于监管提交。
38 敏感数据识别结果: {sensitive_data_result}
39 数据血缘追踪结果: {lineage_result}"
40 )
41
42 # 3. 核心功能函数 (无需修改)
43 def load_data_for_compliance(data_path):
44     """加载合规审计所需数据"""
45     return pd.read_csv(data_path, encoding="utf-8")

```

```

46
47 def identify_sensitive_data(data):
48     """自动识别敏感数据"""
49     data_sample = data.head(200).to_string() # 取前200条样本，确保覆盖所有字段
50     sensitive_result =
51     llm(sensitive_data_prompt.format(data_sample=data_sample))
52     return sensitive_result
53
54 def track_data_lineage(data_info, process_records):
55     """自动梳理数据血缘链路"""
56     lineage_result = llm(data_lineage_prompt.format(
57         data_info=data_info,
58         process_records=process_records
59     ))
60     return lineage_result
61
62 def generate_compliance_report(sensitive_result, lineage_result):
63     """自动生成合规审计报告"""
64     report = llm(compliance_report_prompt.format(
65         sensitive_data_result=sensitive_result,
66         lineage_result=lineage_result
67     ))
68     return report
69
70 # 4. 主函数（修改数据路径和处理记录即可）
71 def main():
72     # 需修改1：待审计数据路径
73     data_path = f"{config.DATA_STORAGE_PATH}/to_audit_data.csv"
74     # 需修改2：数据信息和处理记录（填写数据来源、处理节点等）
75     data_info = "数据名称：企业用户信息表，数据来源：业务系统，数据用途：用户管理与分
76     析"
77     process_records = "处理节点1：数据采集（业务系统同步），处理方式：批量导入；处理节
78     点2：数据清洗（去重、脱敏），处理方式：AI Agent自动处理；处理节点3：数据流转，去向：数据
79     分析部门、运营部门"
80
81     # 执行合规审计流程（无需修改）
82     try:
83         data = load_data_for_compliance(data_path)
84         print("数据加载成功，开始识别敏感数据...")
85         sensitive_result = identify_sensitive_data(data)
86         print("敏感数据识别完成，开始梳理数据血缘...")
87         lineage_result = track_data_lineage(data_info, process_records)
88         print("数据血缘梳理完成，开始生成合规审计报告...")
89         compliance_report = generate_compliance_report(sensitive_result,
90 lineage_result)
91
92         # 保存审计结果和报告

```

```

88         sensitive_output = f"{config.DATA_STORAGE_PATH}/敏感数据识别结果.txt"
89         lineage_output = f"{config.DATA_STORAGE_PATH}/数据血缘追踪结果.txt"
90         report_output = f"{config.DATA_STORAGE_PATH}/合规审计报告.txt"
91
92         with open(sensitive_output, "w", encoding="utf-8") as f:
93             f.write(sensitive_result)
94         with open(lineage_output, "w", encoding="utf-8") as f:
95             f.write(lineage_result)
96         with open(report_output, "w", encoding="utf-8") as f:
97             f.write(compliance_report)
98
99         print("\n=== 合规审计完成 ===")
100        print(f"敏感数据识别结果: {sensitive_output}")
101        print(f"数据血缘追踪结果: {lineage_output}")
102        print(f"合规审计报告: {report_output}")
103    except Exception as e:
104        print(f"合规审计失败, 错误原因: {str(e)}")
105
106    if __name__ == "__main__":
107        main()

```

注意事项

- 处理记录需如实填写数据流转的每一个节点，确保血缘追踪结果准确，满足监管溯源要求；
- 敏感数据识别支持自定义合规规则，可在Prompt中添加行业专属敏感数据类型（如医疗行业的病历信息）；
- 合规审计报告可直接复制提交给监管部门，如需调整格式，可修改Prompt中的报告要求。

第四章：常见问题与避坑指南

4.1 常见问题解决

- 问题1：大模型初始化失败？
解决：检查模型路径是否正确，确保模型文件完整，依赖包版本适配（可重新安装依赖包）。
- 问题2：代码运行时内存不足？
解决：减小批量处理量（修改config.py中的BATCH_PROCESS_SIZE），关闭其他占用内存的程序。
- 问题3：标注/清洗结果准确率低？
解决：调整大模型temperature参数（降低至0.1-0.3），优化Prompt中的标注/清洗规则，增加样本数量。
- 问题4：音频/图像无法正常处理？
解决：检查文件格式是否符合要求，确保音频清晰、图像无损坏，安装对应的处理依赖包。

4.2 避坑指南（重点）

- 避坑1：不盲目追求大模型，中小场景选用Qwen-7B、Llama 3-8B即可，无需使用13B及以上模型，避免资源浪费；
- 避坑2：不修改代码核心逻辑，仅调整数据路径、标注规则等配置项，避免出现运行错误；
- 避坑3：先跑通单一场景，再拓展多场景，例如先落地数据清洗，再尝试标注、合规审计，降低落地难度；
- 避坑4：敏感数据处理后需及时脱敏，避免数据泄露，合规审计报告需定期更新，满足监管要求。

第五章：总结

本手册覆盖AI Agent数据治理全流程，从环境搭建到3大核心场景落地，提供可直接复制的Prompt库和可运行代码库，无需自研、无需复杂配置，小白也能快速上手。

收藏本文，无需再翻阅其他资料，即可快速落地AI Agent数据治理，解决企业数据治理耗时耗力、成本高、合规难的痛点，实现数据治理效率翻倍，挖掘数据核心价值，为企业数字化转型提供支撑。

写这篇实战手册的初衷，是希望能帮大家少走弯路——我知道很多人在落地AI Agent数据治理时，会遇到代码调试卡壳、Prompt优化无果、场景适配困难的问题，明明有工具、有思路，却因为没人指导、没有同行交流，浪费大量时间和精力。

所以，我专门搭建了知识星球付费社群，不是为了盈利，而是想打造一个纯粹的实战交流圈，把手册里没来得及细化的实操细节、代码优化技巧、Prompt定制方法，还有日常工作中遇到的各类疑难问题，都在社群里和大家慢慢分享、一起解决。

社群里不会有夸大的宣传，也不会有无关的打扰，只有实实在在的价值：我会定期更新手册补充内容、分享最新的开源模型适配技巧，也会一对一解答大家在落地过程中遇到的代码、Prompt相关问题；同时，也会邀请社群里的同行一起交流经验，互相借鉴落地案例，避免一个人闭门造车。

如果你在AI Agent数据治理的落地过程中，也有困惑、有疑问，或者想和更多同行交流学习，愿意相信我、和我一起成长，欢迎加入我的知识星球。在这里，我们一起把手册里的方法落地见效，一起解决实际工作中的难题，把数据治理的效率提上去，把踩过的坑都变成经验。

星球入口（可直接点击加入）：【 **大数据资料库·知识星球** 】，加入后可领取手册配套的完整代码包、Prompt优化手册，后续所有更新内容也会第一时间在社群内同步，愿我们都能在AI数据治理的路上，少走弯路、快速成长。

作者微信：bat6188